# DS5216 Artificial Intelligence
# Programming Assignment II

R. D. Y. Vimukthi - DTS2433

## Introduction

Recent advances in computer vision have significantly transformed the analysis of sports videos by enabling automated detection and tracking of players. In cricket, such technologies can support performance analysis, tactical evaluation, and broadcasting enhancements. This project focuses on developing a computer vision pipeline for detecting cricket players in video sequences using a deep learning-based object detection approach.

The project is divided into two main tasks.

- ➢ Task 1: Developing a player detection model using the YOLOv8 (You Only Look Once) framework, a state-of-the-art real-time object detection algorithm.
- ➢ Task 2: Incorporating a pose estimation model similar to OpenPose to identify key body joints of detected players.

## Dataset Preparation

The dataset used in this project consists of short cricket video clips, each ranging between 5 to 10 seconds in duration. A total of 10 video clips were collected and processed for model training and evaluation.

10 cricket videos were downloaded from the publicly available dataset at Kaggle at the link: https://www.kaggle.com/datasets/souravkumarnag/sample-cricket-video-clips.
These videos were spanning for multiple minutes each and contained unnecessary components which did not align with this assignment's objective. Therefore, the dataset was cropped manually into ten videos each spanning for 10 seconds. When selecting video components to crop, following real-world scenarios were considered and included.

- ➢ Videos of both daytime and nighttime matches
- ➢ Players belonging to multiple ethnicities (not biased to a single team)
- ➢ Different quality videos
- ➢ Different types of matches (Test/ IPL/ T20 etc.)
- ➢ Different angle of recording such as:
    - The bowling action and batting focused
    - Boundary focused
    - Audience only
    - A wide area covered with a large number of players
    - Umpire in the frame
    - A single player focused

Once the dataset with above properties is created, Roboflow, an end-to-end computer vision platform (https://roboflow.com/) was used to annotate data with bounding boxes. Each 10 seconds video was converted to 30 frames (3 frames per second) considering the highly

dynamic behaviour of cricket videos to capture changing scenes. Over 300 frames were generated from 10 videos altogether. Each of these frames were manually annotated by drawing a bounding box around players and labeling the class. As only one type of player was used in this assignment, the bounding boxes were labeled as “player” and the remaining space was considered as background. The umpire in the frame was not considered as a player.

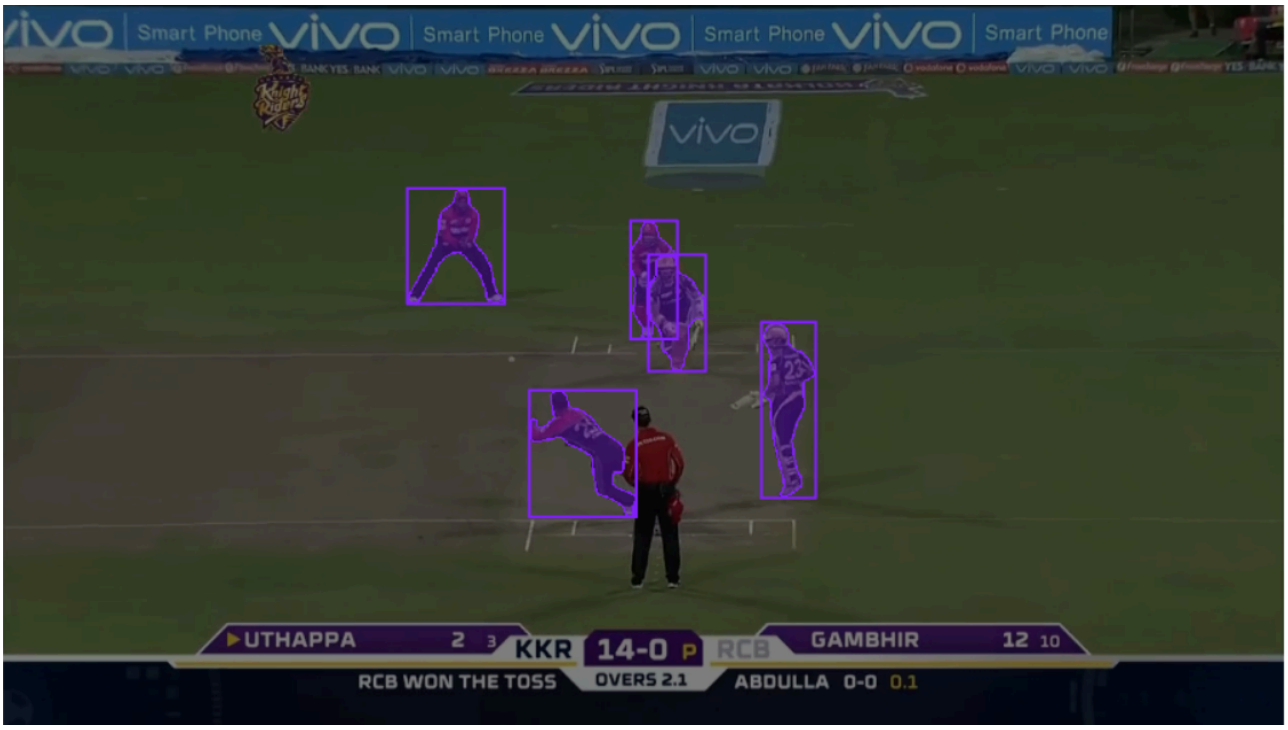


Image 1: Annotating frames with bounding boxes

Once all the frames are annotated, the dataset was split into three parts as train, validation, and test in the ratio of 70:20:10. The training set was used to train the YOLOv8 model, while the validation set was used for hyperparameter tuning and performance monitoring. The test set was used for final evaluation to measure the generalization capability of the model.

The final dataset was structured in YOLO format with corresponding image and label folders for each split.

**Environment Setup and Implementation Details**

The entire project was developed and executed using Google Colaboratory (Google Colab), which provides a cloud-based environment with free access to GPU acceleration. This environment was selected due to its ease of use, preconfigured deep learning support, and compatibility with PyTorch-based frameworks.

- Platform: Google Colab
- Programming Language: Python 3.10+
- Hardware Acceleration: NVIDIA GPU (depending on session availability)
- Deep Learning Framework: PyTorch
- Detection Library: Ultralytics YOLOv8
- Libraries and tools:

- ultralytics – YOLOv8 implementation for object detection
- torch – deep learning backend framework
- opencv-python – image and video processing
- numpy – numerical computations
- pandas – data handling and metric analysis
- matplotlib – visualization of results and training curves
- seaborn – statistical plotting
- glob/os – file handling and dataset management

**Data Preprocessing**

The link to the data preprocessing Colab notebook is here.

The dataset contained randomly selected 215 training images, 61 validation images, and 31 testing images each with its label file containing its class and bounding box coordinates.

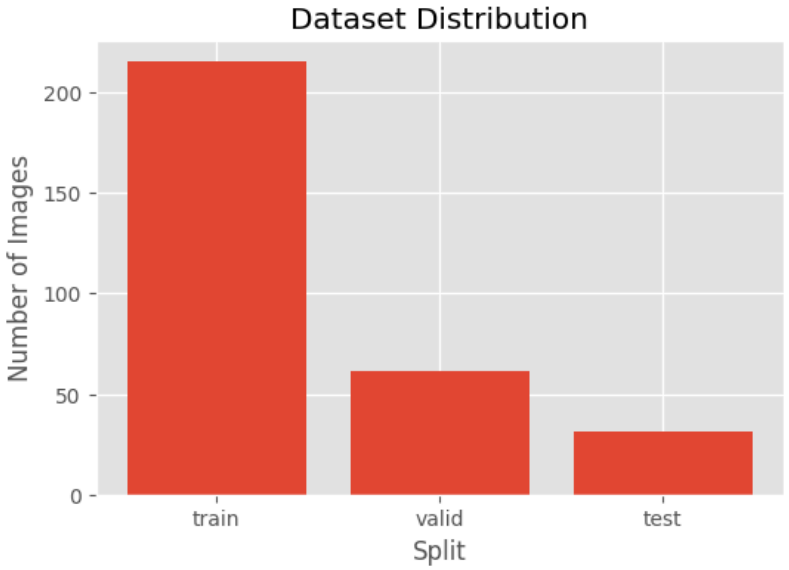


Image 2: Data Distribution across train, valid, and test sets

Image 3: A few random images from the dataset

Additional step: At the annotation stage, for some images, when generating bounding boxes through Roboflow, the annotation type was selected as polygon labels which creates a lot of points in addition to the four corners in our usual bounding boxes. To make executions simpler, these were converted back to traditional annotation bounding box labels with just 4 corners. Therefore, at the end, a label contains 5 values, the class of the object and four corners of the bounding box.

**Model fitting and training**

The YOLOv8 model was selected for this task due to its balance between accuracy and computational efficiency. YOLO is a one-stage object detection framework that performs bounding box regression and classification simultaneously, making it suitable for real-time applications such as cricket player detection. The selected YOLOv8 model was initialized with pretrained weights trained on the COCO dataset. Transfer learning was applied to adapt the model to the cricket player detection task. Following configurations were used in training the model:

- ➢ Image size: 640 × 640 pixels
- ➢ Batch size: 16
- ➢ Number of epochs: 50
- ➢ Optimizer: Default YOLO optimizer (SGD/Adam-based adaptive strategy)

During training, the model optimized bounding box regression loss, classification loss, and distribution focal loss. Training progress was monitored using validation loss and evaluation metrics such as precision, recall, and mean Average Precision (mAP).

| index | epoch | time | train/box_loss | train/cls_loss | train/dfl_loss | metrics/precision(B) | metrics/recall(B) | metrics/mAP50(B) | metrics/mAP50-95(B) | val/box_loss | val/cls_loss | val/dfl_loss | lr/pg0 | l |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 7.3856 | 1.45756 | 2.90198 | 1.12647 | 0.00727 | 0.875 | 0.4516 | 0.28361 | 1.31717 | 3.21148 | 1.06549 | 0.00026 | |
| 1 | 2 | 12.2739 | 1.43987 | 1.80917 | 1.1363 | 0.00743 | 0.89474 | 0.15353 | 0.10377 | 1.29845 | 3.26912 | 1.1134 | 0.000529308 | 0.00 |
| 2 | 3 | 16.5939 | 1.48843 | 1.68833 | 1.22902 | 0.45049 | 0.02176 | 0.04834 | 0.02412 | 1.45018 | 3.39559 | 1.25376 | 0.000787528 | 0.00 |
| 3 | 4 | 20.7952 | 1.4379 | 1.56248 | 1.18778 | 0.33067 | 0.11842 | 0.11116 | 0.05892 | 1.47913 | 3.2004 | 1.24879 | 0.00103466 | 0.0 |
| 4 | 5 | 27.026 | 1.43413 | 1.54219 | 1.18949 | 0.20085 | 0.18189 | 0.10782 | 0.0543 | 1.61771 | 2.97986 | 1.32244 | 0.0012707 | 0. |

Image 4: A few metrics from the training process

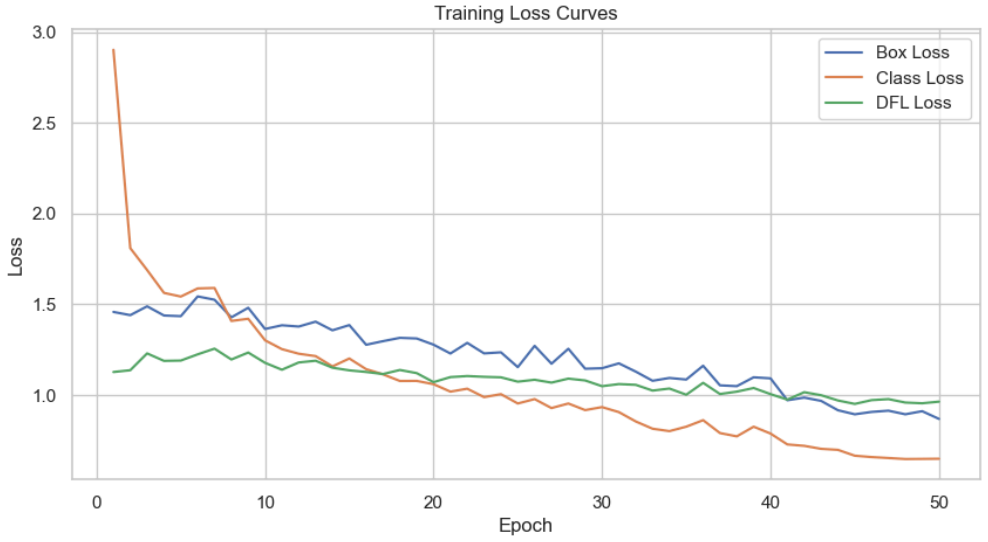


Image 5: Training loss curves (box loss, class loss, DFL loss)

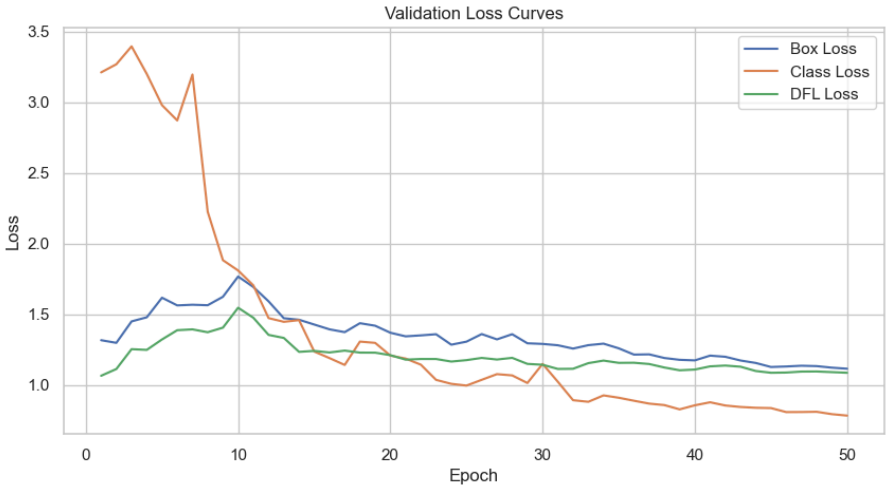


Image 6: Validation loss curves (box loss, class loss, DFL loss)

**Performance Evaluation**

The performance of the model was evaluated using standard object detection metrics (for both validation data and test data). Validation data was used to get an idea about the training process to tune hyperparameters, while test test was used to evaluate model's performance on unseen data.

- ➢ Precision: Measures the proportion of correctly predicted players among all predicted players.
- ➢ Recall: Measures the proportion of actual players correctly detected by the model.
- ➢ mAP@0.5: Mean Average Precision at IoU threshold 0.5, indicating overall detection quality.
- ➢ mAP@0.5:0.95: A stricter evaluation metric that averages performance across multiple IoU thresholds (a comparison of ground truth bounding box and predicted bounding box for detected objects)

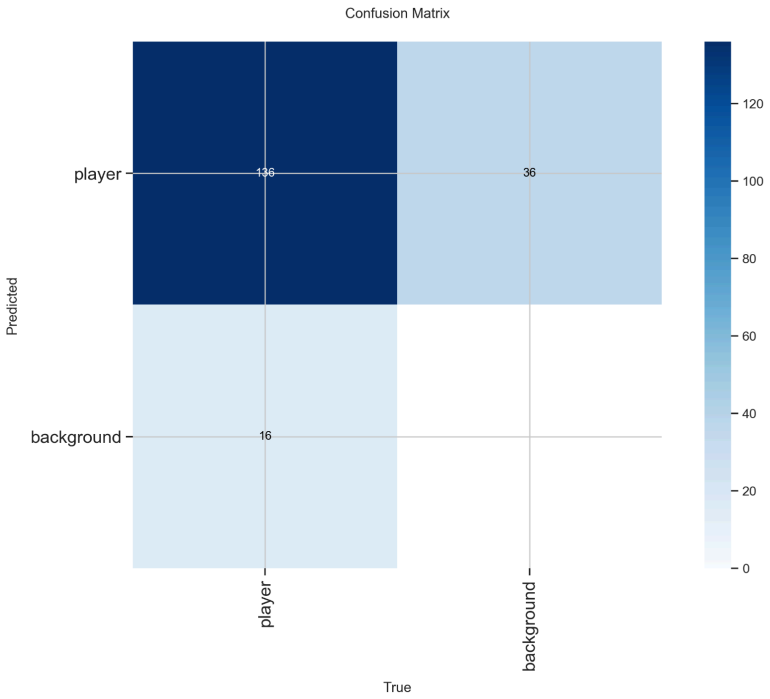


Image 7: Confusion Matrix considering two classes as "player" and "background"

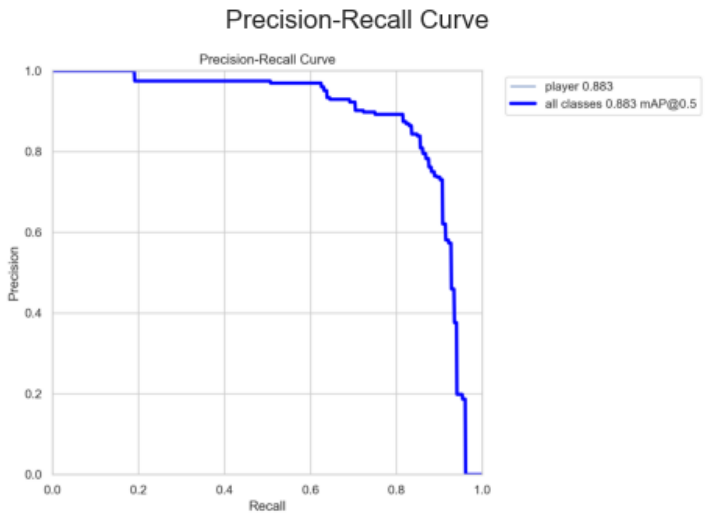


Image 8: Precision- Recall curve

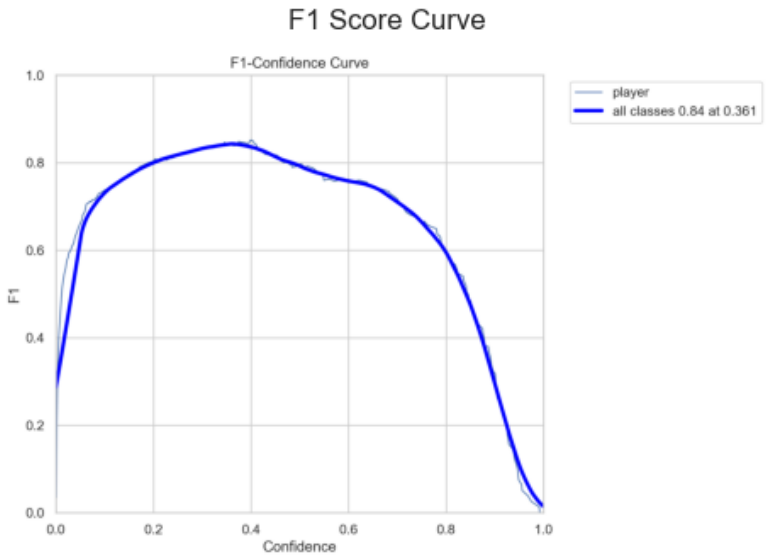


Image 8: F1 Score curve

## Performance Comparison and Insights

### 1. Quantitative Analysis

| | Metric | Validation | Test |
|---|---|---|---|
| 0 | Precision | 0.8427 | 0.8583 |
| 1 | Recall | 0.8487 | 0.7684 |
| 2 | mAP@0.5 | 0.8832 | 0.8854 |
| 3 | mAP@0.5:0.95 | 0.5911 | 0.4954 |

Image 9: Precision, Recall, mAP@0.5, mAP@0.5:0.95 for validation and testing sets

The interpretation for above results are as follows:

- ➢ **Precision:** The model achieved a precision of 0.8427 (validation) and 0.8583 (test). This indicates that most of the detected players are correct, with a relatively low number of false positives. The slightly higher precision in the test set suggests that the model generalizes well to unseen data without over-predicting (overfitting) player instances.
- ➢ **Recall:** The recall values of 0.8487 (validation) and 0.7684 (test) show that the model successfully detects most players in the validation set, but performance decreases in the test set. This drop suggests that some players are missed in more challenging scenarios such as: overlapped/covered players, small or distant players, blur in video frames (specially in old videos with low camera quality).
- ➢ **mAP@0.5:** The model achieved a high mAP@0.5 score of 0.8832 (validation) and 0.8854 (test). This demonstrates that the model is highly effective at detecting and localizing players when a moderate Intersection-over-Union (IoU) threshold is applied. This result confirms that the model is reliable for general player detection tasks in cricket videos.
- ➢ **mAP@0.5:0.95:** Values of 0.5911 (validation) and 0.4954 (test) are significantly lower compared to mAP@0.5. This indicates that while the model performs well in detecting players, its bounding box precision decreases when stricter overlap thresholds are applied.

Overall, these metrics suggest that the model detects players correctly in the majority of instances, however, bounding box tightness will result in lower results. Which means that localization accuracy can be improved further.

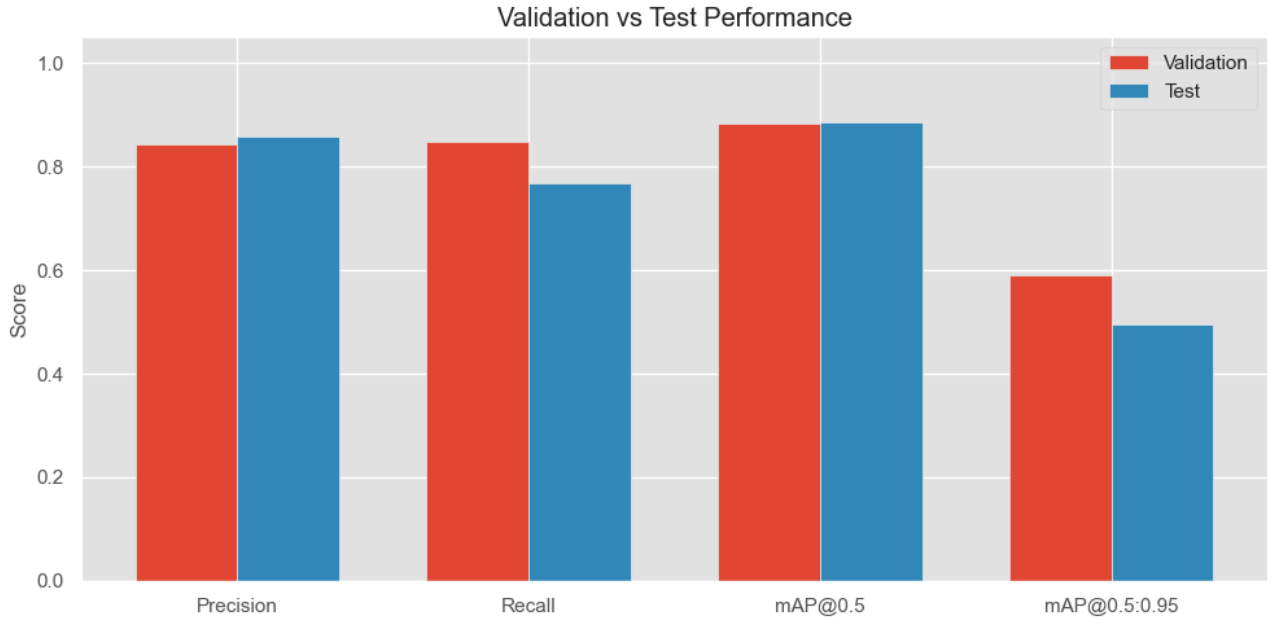


Image 10: Validation vs Test Performance

## 2. Qualitative Analysis

### a. Predictions on images

The model was tested on unseen images extracted from cricket videos. The results show that the model is capable of accurately detecting multiple players in a single frame, even under varying conditions such as different camera angles and distances. The umpire was not categorized as a player, which shows the model capability. Camera crew, extra players outside boundary, or audience were not detected as players. Even in frames with a large number of players (players visible in very small size), almost all players were detected.
In most cases, bounding boxes were tightly aligned with players, and confidence scores were high. However, performance slightly decreased in cases involving occluded or distant players. More output images available here.


Image 11: A few images on test data with player detection

### b. Predictions on videos

To check the real-world usage of the model in detecting players on a live video stream, the model was executed on a set of sample videos. The model was successful in identifying players on the live stream as well. The results demonstrate stable detection across consecutive frames, indicating temporal consistency in predictions. The model is capable of real-time inference, making it suitable for video analysis applications. More output samples available here.


Image 12: Player detection on a video stream

## 3. Limitations

Although the YOLOv8-based model demonstrates strong performance in detecting cricket players, a few limitations were observed based on quantitative and qualitative evaluation results, above. The model shows some drop in recall on the test set (0.7684) compared to validation performance (0.8487). This indicates that the model fails to detect a proportion of players in unseen scenarios. This limitation can be associated with challenging visual conditions such as occlusion, small objects, and blur in fast-moving video frames.

Also, the mAP@0.5:0.95 score (0.4954 on test data) is significantly lower than mAP@0.5, suggesting that while the model is capable of identifying player locations, it struggles with precise bounding box localization under stricter overlap criteria. This shows limitations in detecting players in crowded or distant views.

Additionally, the model was trained as a single-class detector (“player”). The model can be further enhanced to capture different roles among players such as batsmen, baller, fielders, (also umpires), and additional players who are outside the boundary.

Finally, the performance is limited by the relatively small and homogeneous dataset (only 10 short videos). This reduces variability in player appearance, camera angles, and environmental conditions, affecting generalization to more diverse real-world cricket scenarios.

## 4. Error Analysis

One major source of error is false negatives, where the model fails to detect players who are either partially occluded or located far from the camera. These cases are common in cricket fielding positions where players appear very small in the frame. Another observed issue is false positives, where background objects such as audience members, boundary markings, or advertising boards are occasionally misclassified as players. This typically occurs in visually cluttered scenes where object features resemble human shapes. The model also shows inconsistent bounding box quality in certain cases. While most detections are accurate, some bounding boxes are loosely fitted, particularly for fast-moving players or those at the edge of the frame. This contributes to the reduced mAP@0.5:0.95 score. Furthermore, performance degradation is observed in frames with motion blur and rapid camera movement, where visual features become less distinct, leading to missed detections or low-confidence predictions.

Overall, these errors indicate that the model performs best in clear, well-framed scenes but struggles a little under more complex real-world video conditions.

## 5. Possible Improvements

- **Dataset Improvements:** A main improvement would be to increase the size and diversity of the dataset. Including matches from different stadiums, lighting conditions, camera angles, and broadcasting styles would significantly improve generalization (this has been done already, but more data catering to different real-world scenarios will provide better results). The dataset quality can be enhanced by ensuring:
  - Consistent and accurate bounding box annotations
  - Removal of noisy or incorrect labels
  - Better balance between different play scenarios (batting, bowling, fielding phases)

- **Multi-Class Extension:** Instead of treating all players as a single class, the model can be extended into a multi-class detection system. This would include categories such as: Batsmen, Bowler, Fielder, Wicketkeeper, Umpire, and additional players. Such a classification would not only improve analytical depth but also enable richer sports analytics applications such as strategy analysis and player role tracking.

- **Model and Training Improvements:**
  Performance can also be improved by:

- Using larger YOLO variants (YOLOv8m or YOLOv8l) for better feature extraction
- Training for more epochs
- Applying stronger data augmentation techniques such as scaling, motion blur simulation, and brightness variation
- Increasing input resolution to improve detection of small distant players
- Hyperparameter optimization and selecting the optimal set of hyperparameters

---

## Task 2: Player Keypoint Detection

### Details of the model

The keypoint detection component of this system was designed to estimate human body joints of cricket players using a deep learning-based pose estimation approach. After detecting players using a YOLO-based object detection model, each detected bounding box was cropped from the original image and passed to a pose estimation model.

For pose estimation, a pretrained YOLOv8-pose model was utilized. This model is based on a single-stage architecture that performs object detection and keypoint regression together. It predicts 17 standard COCO human keypoints, including:

- ➢ facial landmarks (nose, eyes, ears),
- ➢ upper-body joints (shoulders, elbows, wrists), and
- ➢ lower-body joints (hips, knees, ankles).

The model uses a deep convolutional (lightweight) architecture which helps real-time detection. Since the model is pretrained on the COCO dataset, usually it generalizes well to human poses without requiring additional training on the cricket dataset (however, both pre-trained weights and fine-tuning were tried out).

Option A of implementation includes just applying pre-trained YOLO-pose on a cricket frames dataset. Option B includes fine-tuning the YOLO-pose model on cricket data itself rather than just using pre-trained weights on COCO dataset.

### Evaluation of keypoint detection model

The evaluation of the keypoint detection module was performed using qualitative and quantitative measures. Qualitatively, the system was assessed based on the correctness of skeleton formation, visual alignment of joints with player body structure, and consistency across different cricket scenarios such as batting, bowling, and fielding positions by going through test images. In many scenarios it was observed that keypoints were not generated as a result of players being too small on the frame on the dataset. A few sample outcomes are as follows. More sample outcomes available here.


Image 13: A few sample outcomes from keypoint detection

Quantitatively, evaluation included the average number of detected keypoints per player instance (measure of completeness) and the mean confidence score of predicted joints (measure of reliability). The average detected keypoints per person was 17 and the mean confidence score was 0.69495386

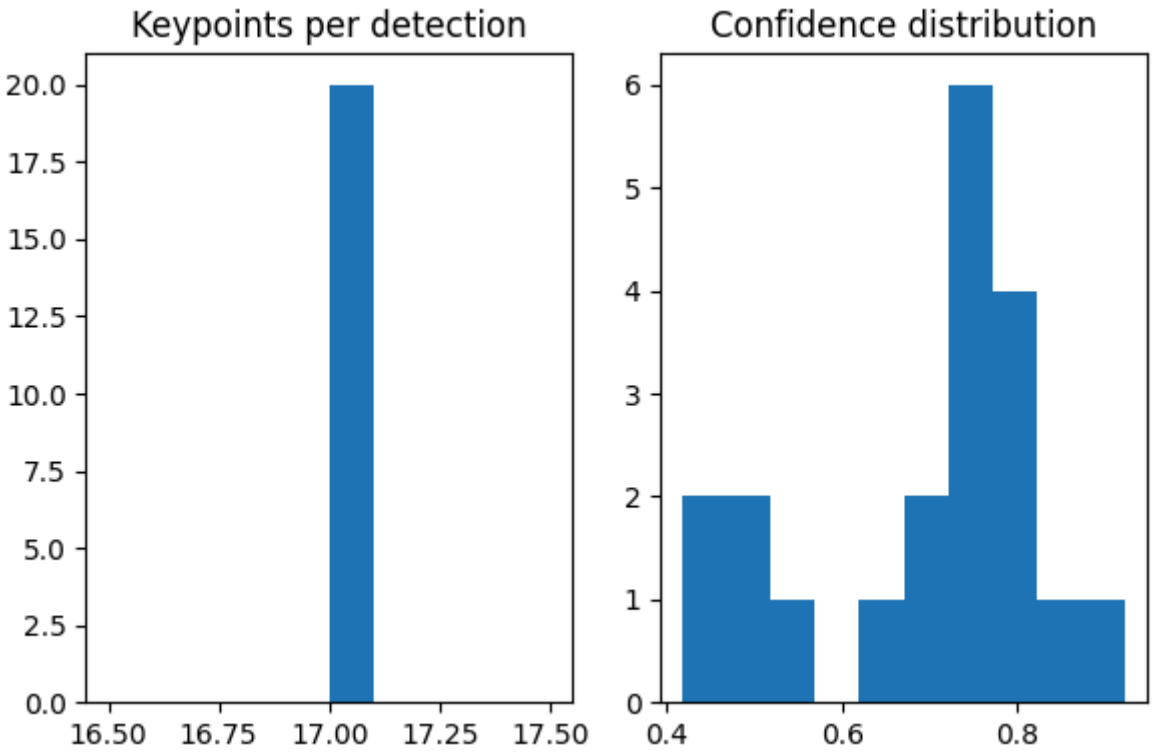


Image 14: Quantitative metrics for keypoint detection

**Insights on model performance**

The model achieved an average of 17.0 keypoints detected per person, which corresponds to the full set of COCO human body keypoints. This indicates that the model is consistently able to detect all standard anatomical joints, including facial landmarks, upper-body joints, and lower-body joints, across the majority of player instances. This result demonstrates that the pose estimation pipeline is structurally complete and capable of producing full-body skeletal representations in cricket scenarios.

The average keypoint confidence score was **0.6949**, indicating a reasonably strong level of prediction certainty. This suggests that most detected joints are predicted with moderate to high confidence, reflecting reliable spatial localization of body parts. However, as shown in image 14, the confidence is not uniformly high across all keypoints, which is expected due to variations in player scale (small players scattered across large ground), occlusion, and blur due to motion in cricket images.

**Observed Limitations**

Although the model was fine-tuned, still the keypoint confidence score is low related to some keypoints. This is mainly due to small objects, heavy occlusion, overlapping players, or distant subjects with low resolution. In some rare instances, audience member keypoints were

detected (when the full frame is on the audience and players are not visible). This is mainly due to the player detection model limitations (an audience member identified as a player and their keypoints generated). In some instances with multiple players on the frame, only one player's keypoints were generated although all players are identified through player detection.

**Possible Improvements**

A major improvement would be to fine-tune the model further with a larger cricket dataset. Manually annotating keypoints and detecting would provide better results, however, it is a heavily time-consuming task. Therefore, the ideal option would be to use a pre-trained pose model trained on COCO dataset and fine-tune it on a good quality cricket dataset. Multi-object tracking algorithms such as DeepSort can also be applied to ensure consistent player identification across frames. Cricket specific keypoints such as bat position, bowling arm angle, etc. can also be incorporated to match real-world cricket-specific use cases.

---

**Important Links:**

1. Folder containing train, test, valid data for player detection model (also contains notebook files)
2. Folder with output samples with players detected on test data (frames) and video streams
3. Keypoint detection dataset and colab notebook
4. Keypoint detection output sample with detected (and labelled) keypoints on test frames
5. Link to the GitHub Repository